CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

System Design Document (SDD)

Project Title: RentEasy

Prepared By: Nuevas, Josh Anton K.

Version: 0.2

Date: 02/28/2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/21/2026	Josh Anton K. Nuevas	Initial draft	Draft
0.2	02/28/2026	Josh Anton K. Nuevas	Added contents for System Architecture, API Contract and Communication Flow, Database Design, UI/UX Design, and Project Plan.	Final
0.3	[Date]	[Your Name]	Updated database design	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY

1.1 Project Overview

RentEasy is a specialized e-commerce marketplace designed to facilitate the peer-to-peer or business-to-consumer rental of items. Unlike traditional sales platforms, RentEasy focuses on temporal ownership, allowing users to rent high-value equipment such as cameras and tools through a unified digital interface.

1.2 Objectives

1. Architectural Integrity: Implement a robust three-tier architecture utilizing Spring Boot for the backend, React for the web interface, and Kotlin/Jetpack Compose for the mobile experience.
2. Interoperability: Develop standardized RESTful APIs to ensure seamless data synchronization across all platforms.
3. User Experience: Deliver a responsive, high-performance UI tailored for both desktop and native Android environments.

1.3 Scope

Included Features:

- User registration and authentication (email/password)
- Rental item listing with search functionality
- Shopping cart management (add, remove, update quantities)
- Checkout process with shipping information collection
- Order placement with confirmation
- Admin panel for rental item and inventory management
- Responsive web interface
- Native Android mobile application

Excluded Features:

- Real payment gateway integration
- Product reviews and ratings system
- Inventory management automation
- Email notification system

- Social media login integration
- Push notifications
- Advanced analytics dashboard

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the RentEasy system. It provides detailed requirements, architectural decisions, API contracts, database design, and an implementation roadmap to guide development and ensure all components integrate seamlessly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: RentEasy
Domain: E-commerce/Rental Services
Primary Users:

1. Customers
2. Administrators

Problem Statement: Users require a streamlined, efficient method to browse and rent high-value items without the overhead of large-scale e-commerce platforms.
Solution: A minimalist rental marketplace focusing on core booking functionality with a consistent experience across web and mobile.

2.2 Core User Journeys

Journey 1: First-time Customer Rental

1. User visits web application.
2. Clicks "Sign Up" and creates account
3. Browses rental item listing
4. Clicks item to view details
5. Adds item to rental cart
6. Views cart and proceeds to checkout

7. Enters delivery information
8. Places order and receives confirmation

Journey 2: Returning Customer Rental

1. User logs in with existing credentials
2. Searches for a specific rental item
3. Adds multiple items to cart
4. Updates rental quantities
5. Completes checkout using saved information
6. Receives order confirmation

Journey 3: Administrator Inventory Management

1. Admin logs in with admin credentials
2. Navigates to product management section
3. Adds new rental item with details
4. Edits existing item information
5. Views list of all active rentals
6. Manages item inventory levels

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout)
2. Rental catalog (list, detail, search)
3. Shopping cart (add, remove, update, view)
4. Checkout process (shipping, order placement)
5. Admin panel (product CRUD, order view)

SHOULD HAVE

1. Product categories and filtering
2. Order history for users
3. Basic input validation feedback
4. Responsive design for all screen sizes

COULD HAVE

1. Product wishlist
2. Basic sales dashboard
3. Order status tracking

WON'T HAVE

1. Real payment processing
2. Email notifications
3. Social login
4. Advanced analytics
5. Multi-language support

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password
- **Fields:** Email, Password, Confirm Password
- **Validation:** Email format, password strength, uniqueness
- **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout
- **Security:** JWT tokens, password hashing with bcrypt

Feature: Rental Catalog

- **Screens:** Item Listing, Item Detail
- **Display:** Grid layout, item images, names, rental prices
- **Search:** By item name
- **API Endpoints:** GET /products, GET /products/{id}, GET /products/search
- **Admin Functions:** POST /products, PUT /products/{id}, DELETE /products/{id}

Feature: Shopping Cart

- **Screens:** Cart View
- **Functions:** Add item, remove item, update quantity
- **Persistence:** Database storage per user
- **API Endpoints:** GET /cart, POST /cart/items, PUT /cart/items/{id}, DELETE /cart/items/{id}

Feature: Checkout Process

- **Screens:** Checkout Form, Order Confirmation
- **Data Collected:** Shipping address (name, address, city, zip)
- **Process:** Validate input, create order, clear cart
- **API Endpoints:** POST /orders, GET /orders/{id}

Feature: Admin Panel

- **Screens:** Product Management, Order List.
- **Functions:** Add/edit/delete items, view orders.
- **Access Control:** Admin role required.
- **API Endpoints:** Admin-prefixed endpoints with role validation.

2.5 Acceptance Criteria

AC-1: Successful User Registration

- Given I am a new user
- When I enter a valid email and a strong password
- And confirm that the password matches
- And click "Create Account"
- Then my account should be successfully created
- And I should be automatically logged in
- And redirected to the homepage
- And a confirmation email should be queued (optional future feature)

AC-2: User Login

- Given I am a registered user
- When I enter correct credentials
- Then I should be authenticated and receive an access token
- And I should be redirected to my home page
- And invalid credentials should show a clear error message

AC-3: Rental Order Flow

- Given I am logged in as a customer

- When I add an item to my cart
- And proceed to checkout
- And enter valid shipping information
- And place the order
- Then I should see an order confirmation screen
- And my cart should be emptied
- And the order should appear in the admin panel

AC-4: Admin Product Management

- Given I am logged in as an administrator
- When I add a new rental item with valid details
- And save the item
- Then the item should appear in the customer catalog
- And be available for rental
- And editing or deleting the item should update or remove it in the catalog accordingly

AC-5: Shopping Cart Management

- Given I am a logged-in customer
- When I add items to my cart
- Then each item should be reflected in the cart with correct quantity and price
- And I can update quantities or remove items
- And total price should update accordingly

AC-6: Checkout Validation

- Given I am proceeding to checkout
- When I enter invalid shipping information
- Then I should see clear validation messages
- And the order should not be submitted until corrected

AC-7: Order History & Tracking

- Given I am a logged-in customer
- When I view my order history
- Then I should see a list of past orders with details: items, quantity, total price, status

AC-8: Admin Order Management

- Given I am logged in as an administrator
- When I view orders
- Then I should see all customer orders with full details
- And I can update order status (e.g., Processing → Shipped)

AC-9: Security & Authentication

- Given I attempt to access protected endpoints
- When I provide an invalid or expired JWT
- Then I should receive a 401 Unauthorized error
- And access should be denied until proper authentication

AC-10: Search & Filtering

- Given I am a customer browsing the catalog
- When I search by item name or apply filters
- Then only items matching the criteria should appear
- And results should update dynamically

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- Average API response time: ≤ 2 seconds for 95% of requests
- Maximum API response time: ≤ 5 seconds under high load
- Web page load time: ≤ 3 seconds on broadband connections
- Mobile app cold start time: ≤ 3 seconds
- Concurrent users supported: ≥ 100 simultaneous users
- Database query execution: ≤ 500ms for standard queries
- File upload/download time: ≤ 5 seconds for standard item images (<5MB)

3.2 Security Requirements

- Data transport: HTTPS for all communications
- Authentication: JWT token-based authentication for all protected endpoints
- Password storage: bcrypt hashing with 12 salt rounds
- Vulnerability protection: SQL injection, XSS, CSRF prevention on all input fields
- Access control: Role-based access (Customer, Admin) with admin endpoints restricted
- Session management: Tokens expire after 60 minutes; refresh tokens required for renewal
- Logging: Sensitive data excluded from logs; access logs maintained for auditing

3.3 Compatibility Requirements

- Web browsers: Chrome, Firefox, Safari, Edge (latest 2 versions)
- Android: API Level 24+ (Android 7.0+)
- Screen sizes supported: Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- Operating systems: Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+
- Backend API: Fully compatible with REST clients and Android Retrofit library
- Frontend: Responsive design for all supported screen sizes

3.4 Usability Requirements

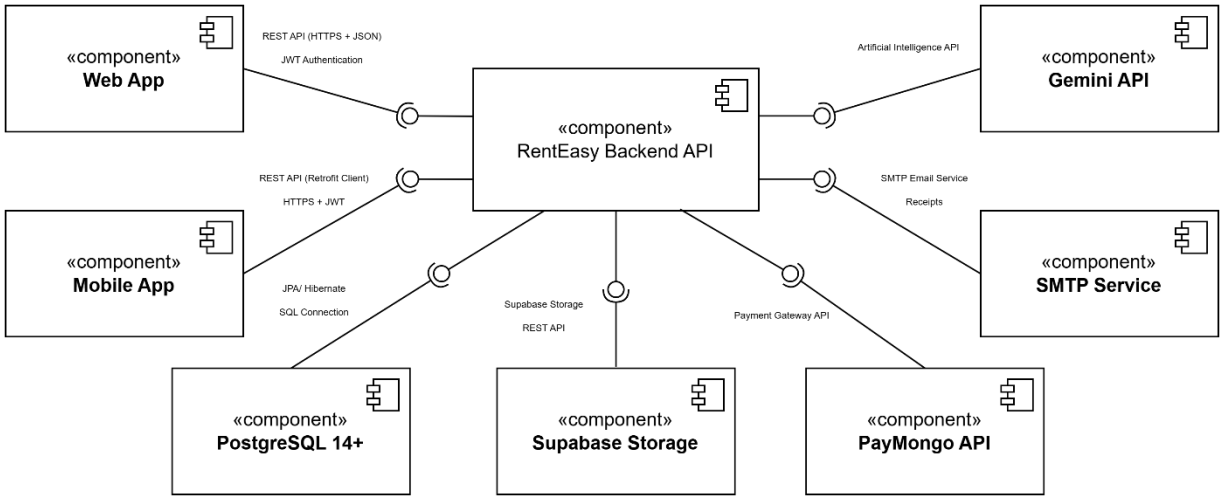
- Tenant onboarding: Complete account registration and first rental request within 5 minutes
- Accessibility: WCAG 2.1 Level AA compliance for web interfaces

- Navigation: Consistent menus, buttons, and dashboard experience across web and mobile
- Error handling: Clear, actionable messages with guidance for recovery
- Mobile interaction: Touch targets minimum 44x44px, smooth scrolling and responsive gestures
- Feedback: Loading indicators for actions like checkout, search, and image uploads
- User satisfaction: Intuitive workflows to minimize clicks needed for major actions

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend**: Java 17, Spring Boot 3.5.11, Spring Security, and Spring Data JPA.
- **Database**: PostgreSQL 14+ (hosted via Supabase).
- **Web Frontend**: React 18 and TypeScript for a type-safe, responsive interface.
- **Mobile**: Kotlin, Retrofit for API consumption, and Room for local data persistence.
- **Build Tools**: Maven (Backend), npm (Web), and Gradle (Android).
- **Deployment**: Railway/Heroku for the backend, Vercel/Netlify for the web frontend, and APK distribution for mobile.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://api.renteasy.com/api/v1
Format	JSON is utilized for all requests and responses.
Authentication	Bearer token (JWT) in Authorization header.

Response Structure	{
	"success": boolean,
	"data": object|null,
	"error":
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": <email>,
	"password": <password>,
	"firstname": <firstname>,
	"lastname": <lastname>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"user": {
	"email": <email>,
	"firstname": <firstname>,
	"lastname": <lastname>
	},
	"accessToken": <token>,

	"refreshToken": <token>
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": <email>,
	"password": <password>
	}

Response Structure	{
	"success": boolean,
	"data": {
	"user": {
	"email": <email>,
	"firstname": <firstname>,
	"lastname": <lastname>,
	"role": <role>
	},
	"accessToken": <token>,
	"refreshToken": <token>
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

User Logout

Description	User Logout

API URL	/auth/logout
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Required

Request Payload	{
	"refreshToken": "<refreshToken>"
	}

Response Structure	{
	"success": true,
	"data": null,
	"error": null,
	"timestamp": "2026-02-21T10:30:00Z"
	}

Product Endpoints

Get All Products

Description	Get All Products
API URL	/products
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Optional
Request Payload	None

Response Structure	{
	"success": true,
	"data": [
	{
	"productID": 1,
	"name": "Camera X100",
	"description": "High-quality DSLR",
	"price": 200,
	"stock": 5,
	"category": "Cameras",
	"image": "https://..."
	}
	],
	"error": null,
	"timestamp": "2026-02-21T10:30:00Z"
	}

Get Product by ID

Description	Get Product by ID

API URL	/products/{id}
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Optional
Request Payload	None

Response Structure	{
	"success": true,
	"data": {
	"productID": 1,
	"name": "Camera X100",
	"description": "High-quality DSLR",
	"price": 200,
	"stock": 5,
	"category": "Cameras",
	"image": "https://..."
	},
	"error": null,
	"timestamp": "..."
	}

Search Products

Description	Search Products
API URL	/products/search?query=<name>
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Optional
Request Payload	None
Response Structure	None

Cart Endpoints

Get User Cart

Description	Get User Cart
API URL	/cart
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Required
Request Payload	None

Response Structure	{
	"success": true,
	"data": {
	"cartID": 101,

	"items": [
	{
	"cartItemID": 1,
	"productID": 1,
	"name": "Camera X100",
	"quantity": 2,
	"price": 200
	}
	],
	"total": 400
	},
	"error": null,
	"timestamp": "2026-02-21T10:30:00Z"
	}

Add Item to Cart

Description	Add Item to Cart
API URL	/cart/items
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Required

Request Payload	{
	"productID": 1,
	"quantity": 2
	}

Response Structure	Returns updated cart

Update Cart Item

Description	Update Cart Item
API URL	/cart/items/{cartItemID}
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Required

Request Payload	{
	"quantity": 3
	}

Response Structure	Updated cart item quantity

Remove Item from Cart

Description	Remove Item from Cart
API URL	/cart/items/{cartItemID}

HTTP Method	DELETE
Format	JSON for all requests/responses
Authentication	Required
Request Payload	None
Response Structure	Updated cart

Cart Endpoints

Place Order

Description	Place Order
API URL	/orders
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Required

Request Payload	{
	"shippingAddress": {
	"name": "Josh Anton",
	"address": "123 Main St",
	"city": "Quezon City",
	"zip": "1100"
	}
	}

Response Structure	{
	"success": true,
	"data": {
	"orderID": 555,
	"orderNumber": "ORD-20260228-001",
	"total": 400,
	"status": "Processing"
	},
	"error": null,
	"timestamp": "2026-02-21T10:30:00Z"
	}

Get Order by ID

Description	Get Order by ID
API URL	/orders/{id}
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Required

Request Payload	None
Response Structure	Detailed order information

Admin Endpoints

Add Products

Description	Add Products
API URL	/admin/products
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Required

Request Payload	{
	"name": "Camera X200",
	"description": "Professional DSLR camera",
	"price": 300,
	"stock": 10,
	"category": "Cameras",
	"image": "https://..."
	}

Response Structure	Created product details

Update Products

Description	Update Products
API URL	/admin/products/{id}
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Required
Request Payload	Updated fields
Response Structure	Updated product details

Delete Products

Description	Delete Products
API URL	/admin/products/{id}
HTTP Method	DELETE
Format	JSON for all requests/responses
Authentication	Required
Request Payload	None
Response Structure	Success confirmation

View All Products

Description	View All Products
API URL	/admin/orders
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Required
Request Payload	None
Response Structure	List of all orders, each with full order structure

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	{
	"success": false,
	"data": null,
	"error": {
	"code": "BUSINESS-002",
	"message": "Property unavailable",
	"details": "This property is no longer available for new rental requests."
	},
	"timestamp": "2026-02-21T10:30:00Z"
	}

Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",

	"message": "Validation failed",
	"details": {
	"startDate": "Start date is required",
	"leaseDurationMonths": "Lease duration must be at least 1 month"
	}
	},
	"timestamp": "2026-02-21T10:30:00Z"
	}

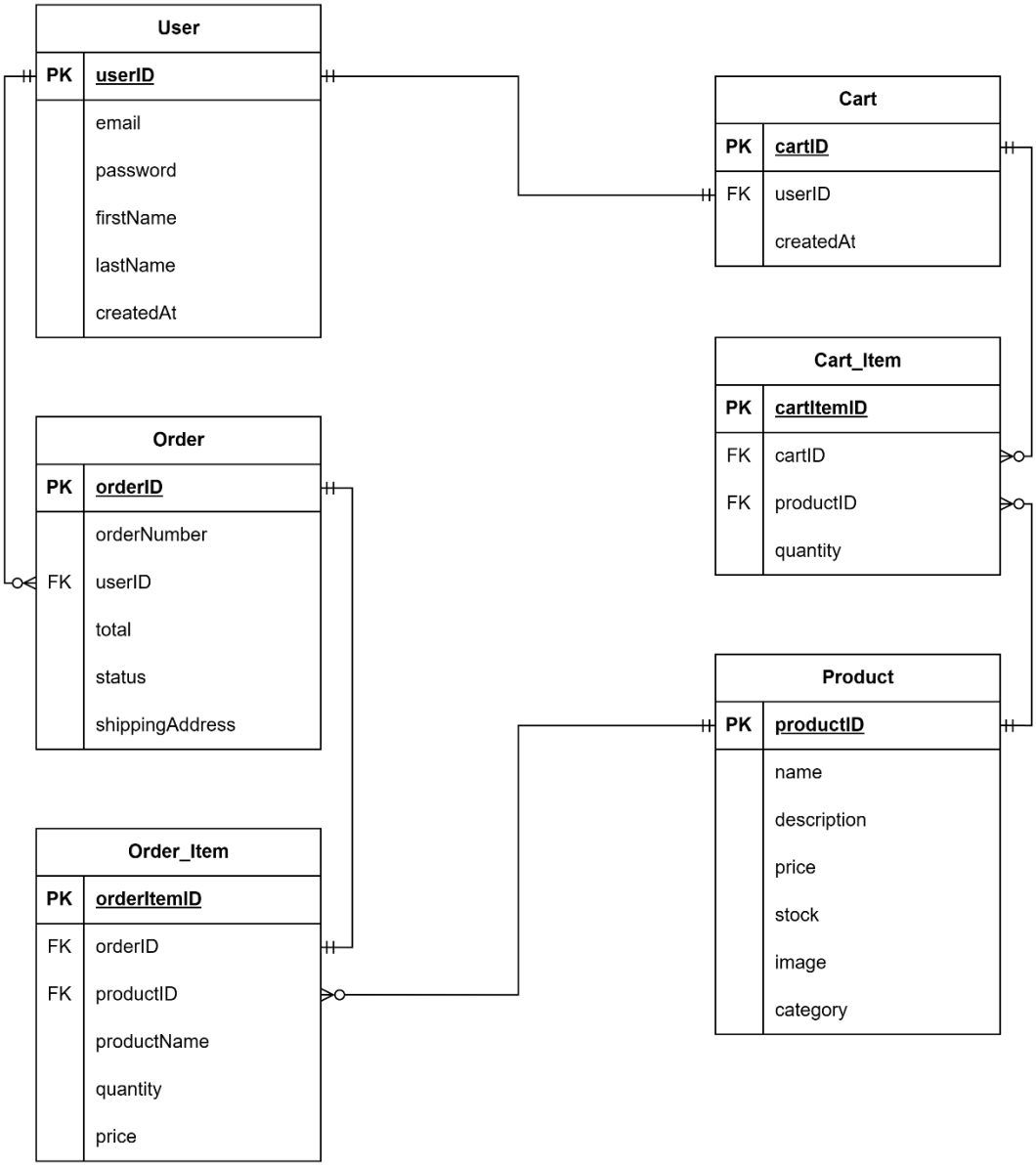
Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Rental request cannot be processed
- BUSINESS-002: Property is no longer available
- PAYMENT-001: Payment failed
- PAYMENT-002: Payment verification failed
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)



Detailed Relationships:

- One-to-One: User to Cart: Each user is assigned exactly one active shopping cart for pending rental items.
- One-to-Many: User to Orders: A user can initiate multiple rental transactions over time.
- One-to-Many: Cart to CartItems: A single cart can contain multiple unique products intended for rental.
- One-to-Many: Order to OrderItems: Once an order is placed, it tracks multiple items as individual line entries.
- Many-to-One: CartItems/OrderItems to Product: Multiple separate rental instances can reference a single product from the primary catalog.

6.2 Key Tables

- User: Stores core account information and authentication credentials.
- Product: Contains the comprehensive catalog of rental items, including pricing and availability.
- Cart: Acts as a temporary session-based container for a user's selected items.
- Cart_Item: Junction table linking specific products and quantities to a user's cart.
- Order: Records finalized rental transactions and shipping logistics.
- Order_Item: Captures the state of products (name, price) now of order placement.

Table Structure Summary:

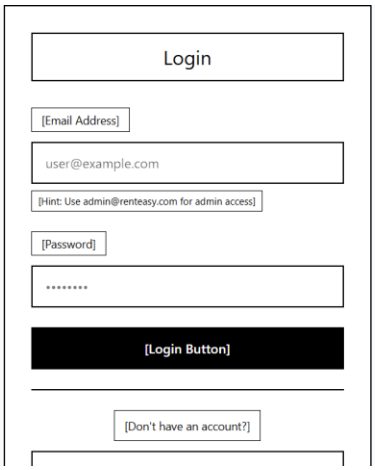
- User: userID, email, password, firstName, lastName, createdAt
- Product: productID, name, description, price, stock, image, category
- Cart: cartID, userID, createdAt
- Cart_Item: cartItemID, cartID, productID, quantity
- Order: orderID, orderNumber, userID, total, status, shippingAddress
- Order_Item: orderItemID, orderID, productID, productName, quantity, price

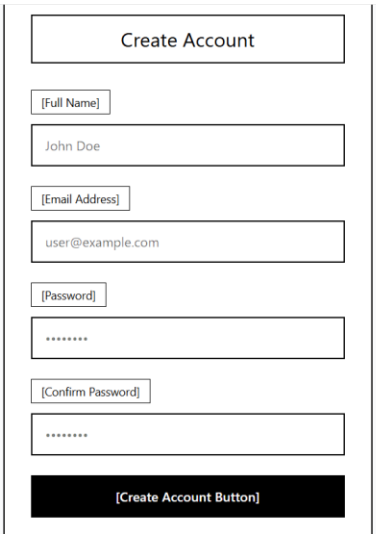
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma

Login and Register Page



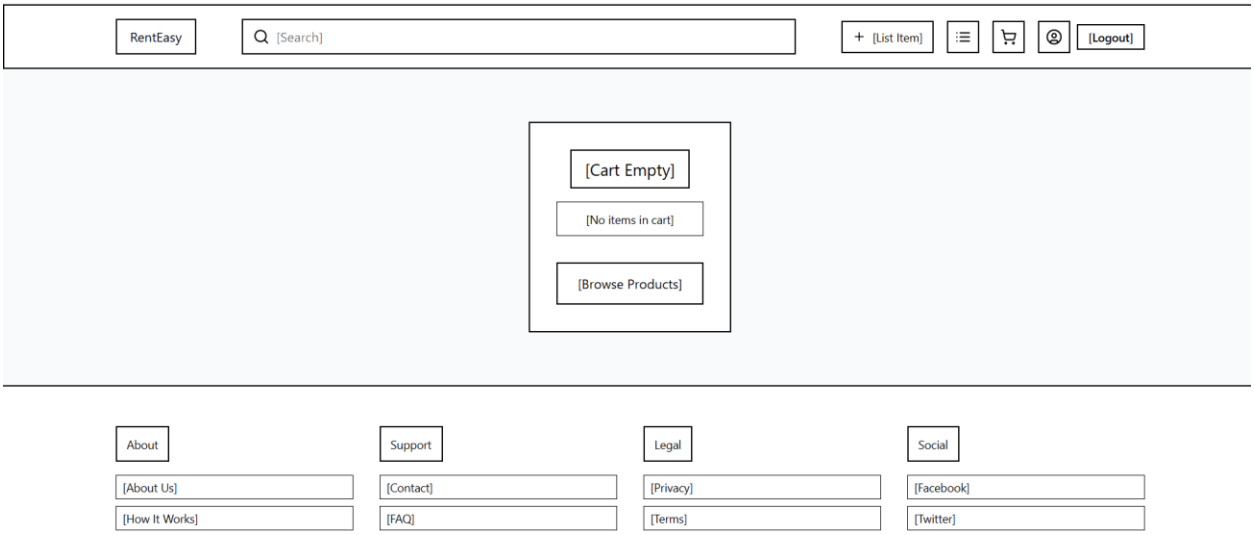


User Home Page

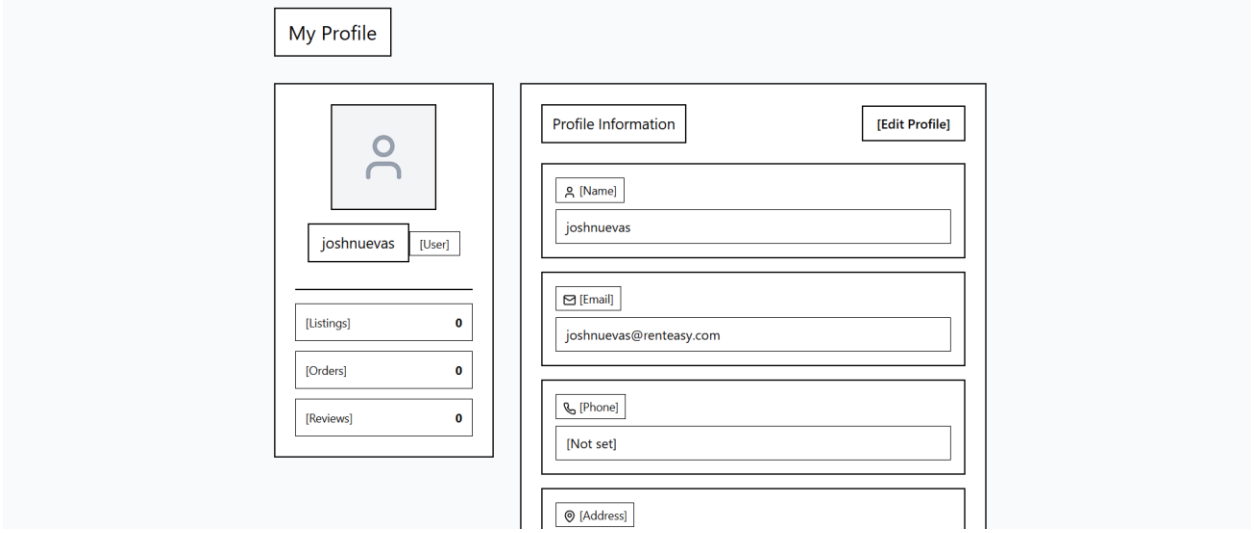
List Item Page

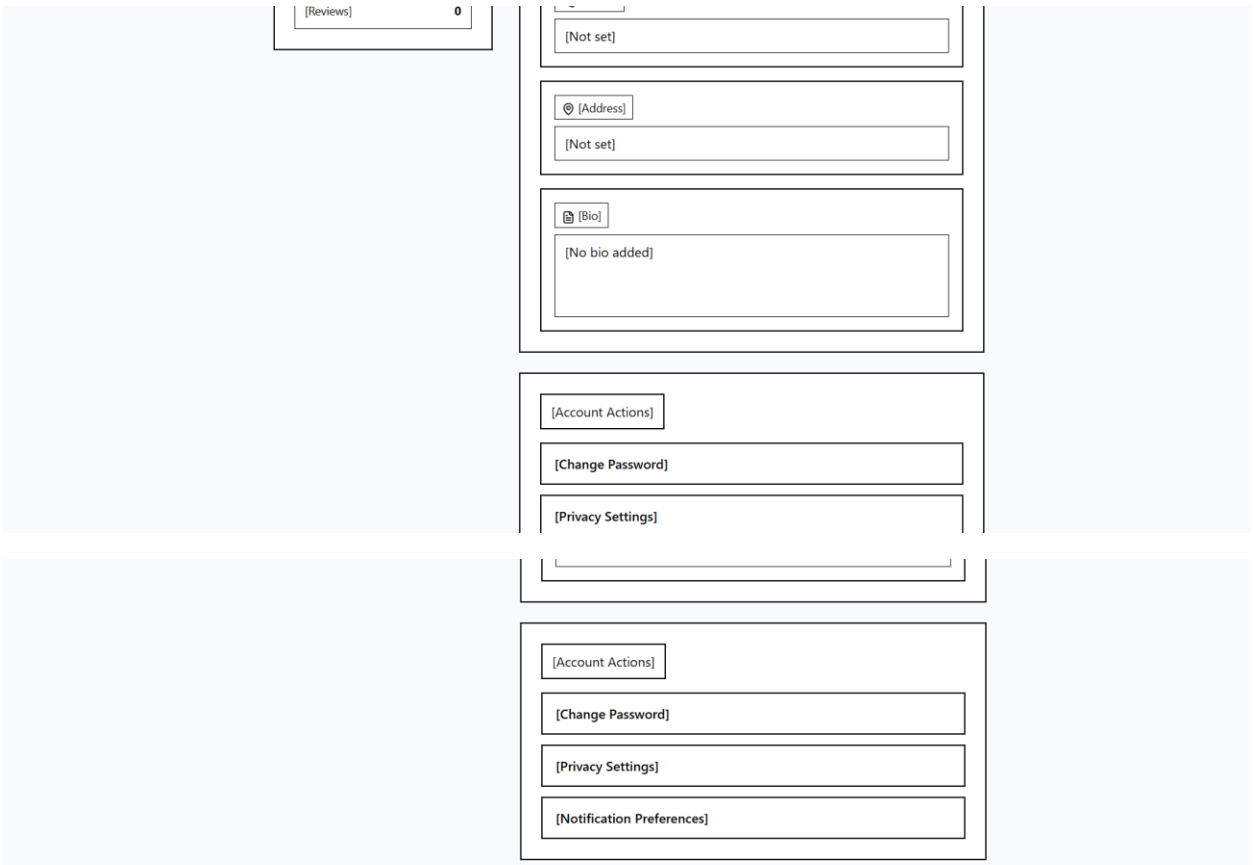
Create Listing Page

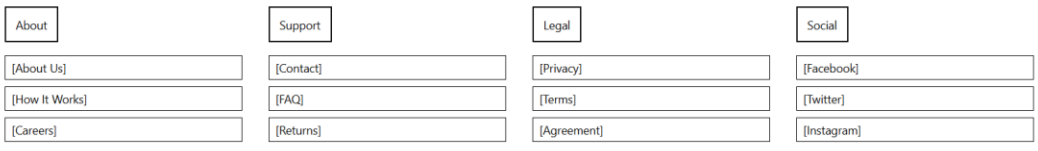
Cart Page



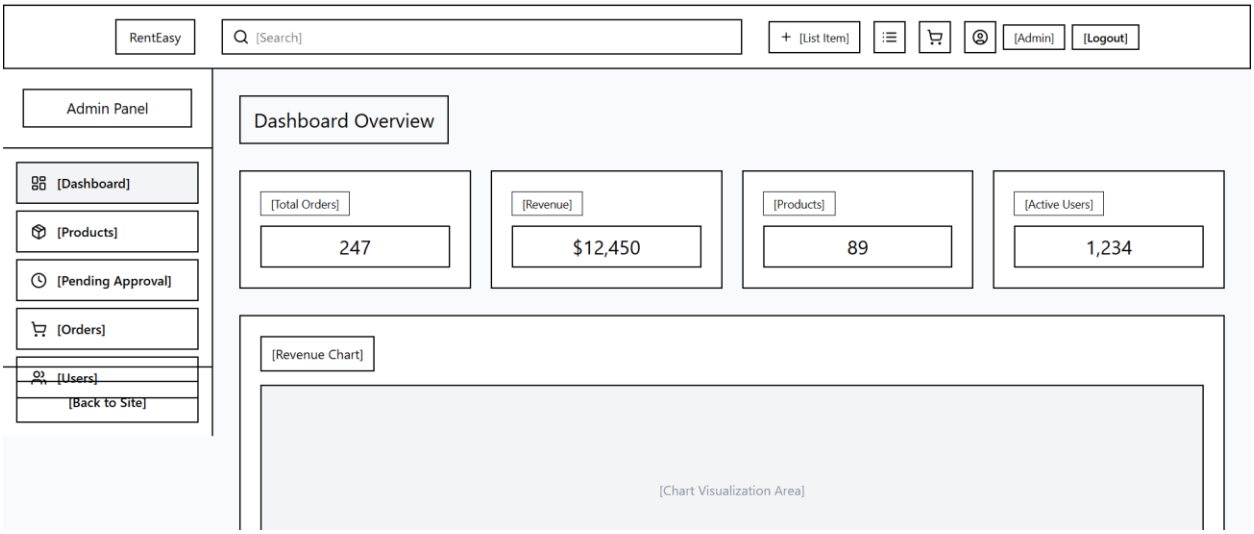
User Profile Page



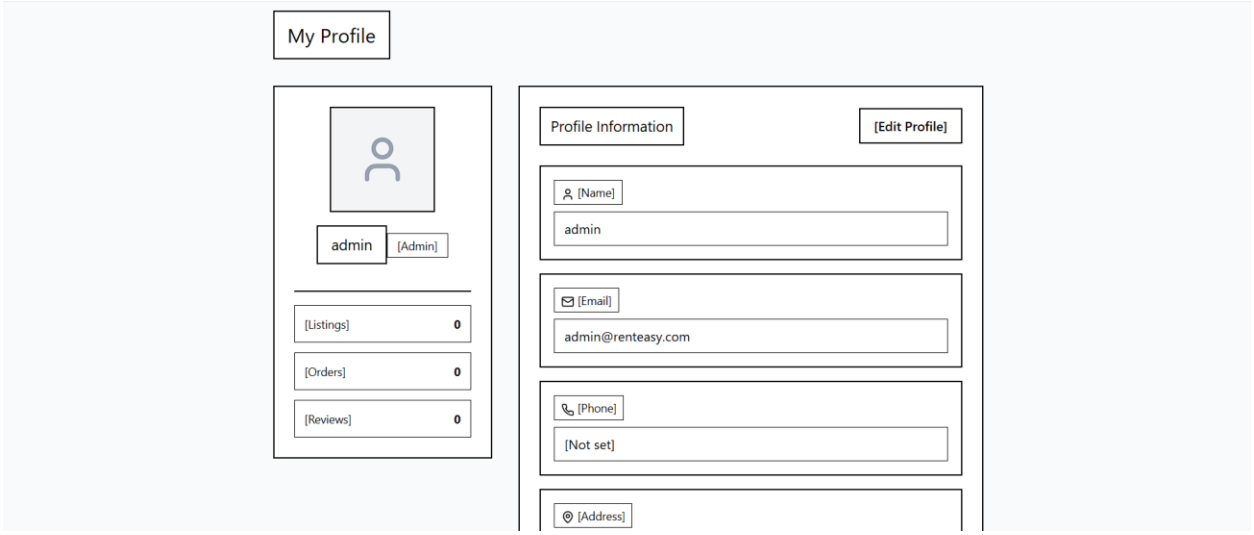

[Reviews] 0
[Not set]
[Address]
[Not set]
[Bio]
[No bio added]
[Account Actions]
[Change Password]
[Privacy Settings]
[Account Actions]
[Change Password]
[Privacy Settings]
[Notification Preferences]


About
[About Us]
[How It Works]
[Careers]
Support
[Contact]
[FAQ]
[Returns]
Legal
[Privacy]
[Terms]
[Agreement]
Social
[Facebook]
[Twitter]
[Instagram]

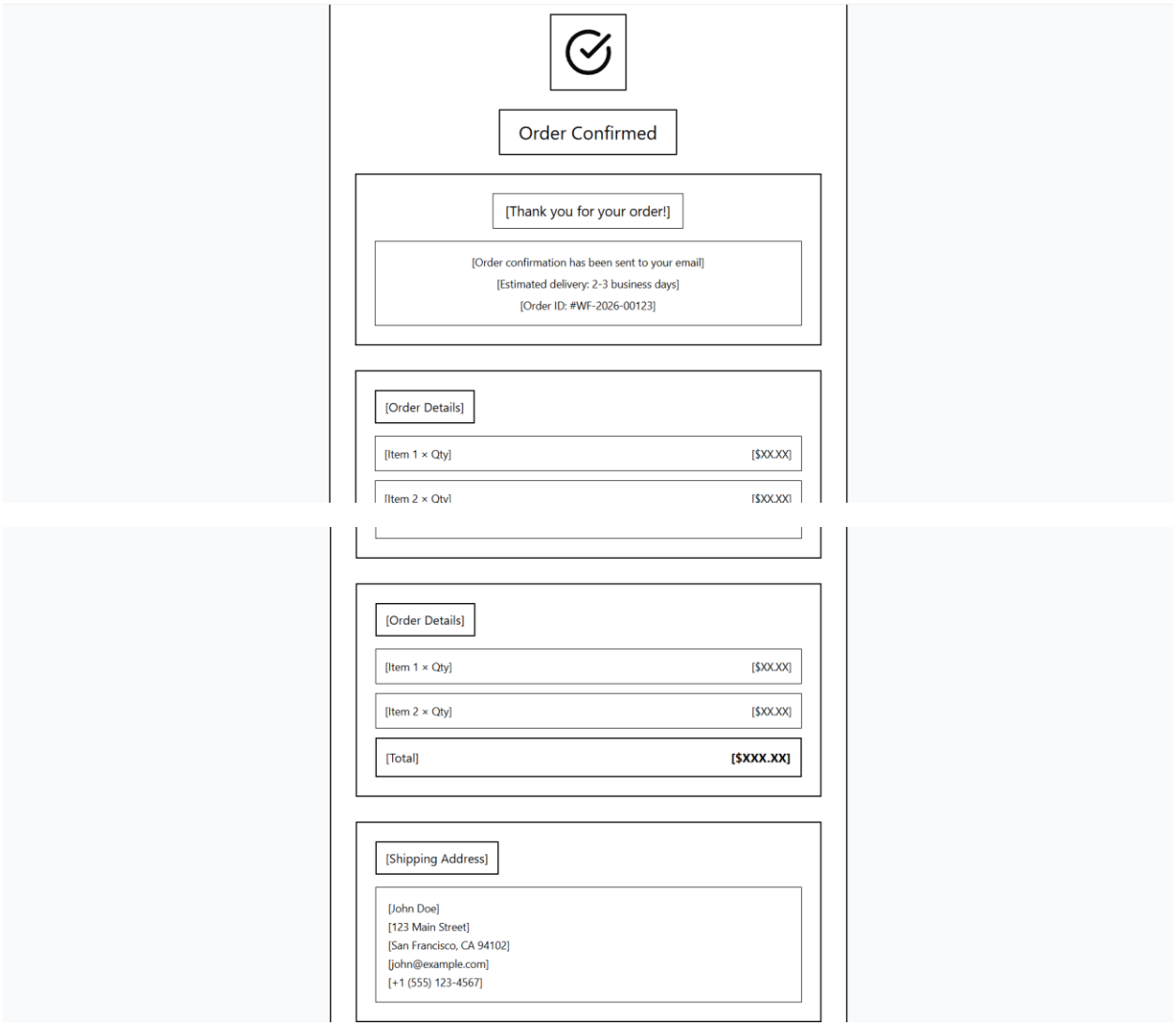
Admin Home Page



Admin Profile Page



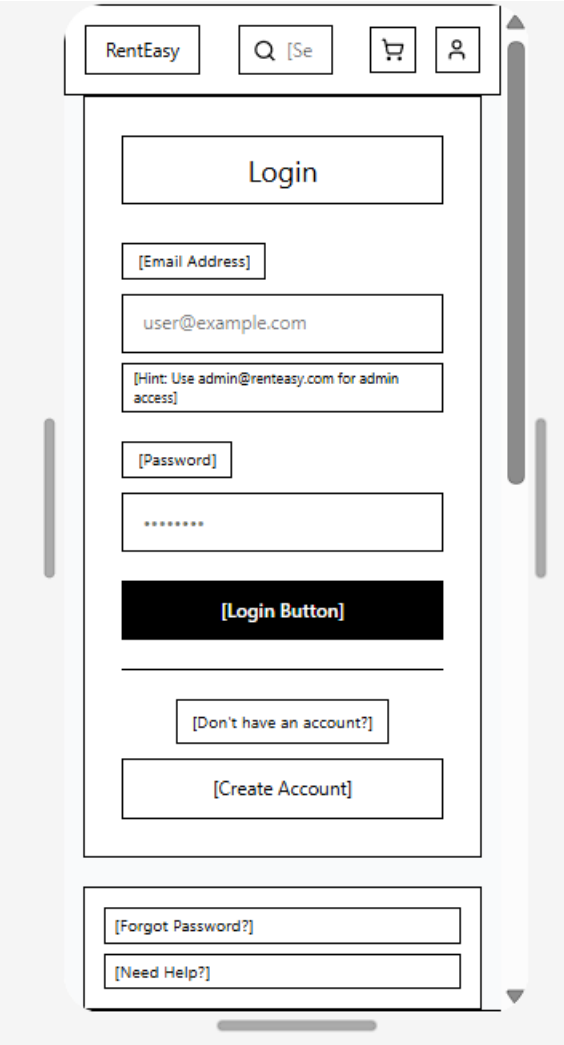
Payment Process Page

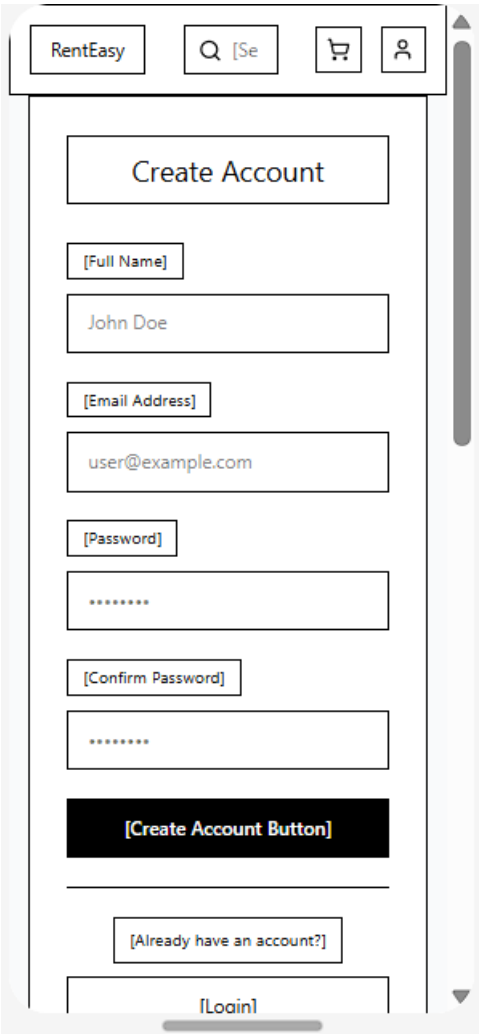

Order Confirmed
[Thank you for your order!]
[Order confirmation has been sent to your email]
[Estimated delivery: 2-3 business days]
[Order ID: #WF-2026-00123]
[Order Details]
[Item 1 × Qty]
[$XX.XX]
[Item 2 × Qty]
[$XX.XX]
[Total]
[$XXX.XX]
[Shipping Address]
[John Doe]
[123 Main Street]
[San Francisco, CA 94102]
[john@example.com]
[+1 (555) 123-4567]

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

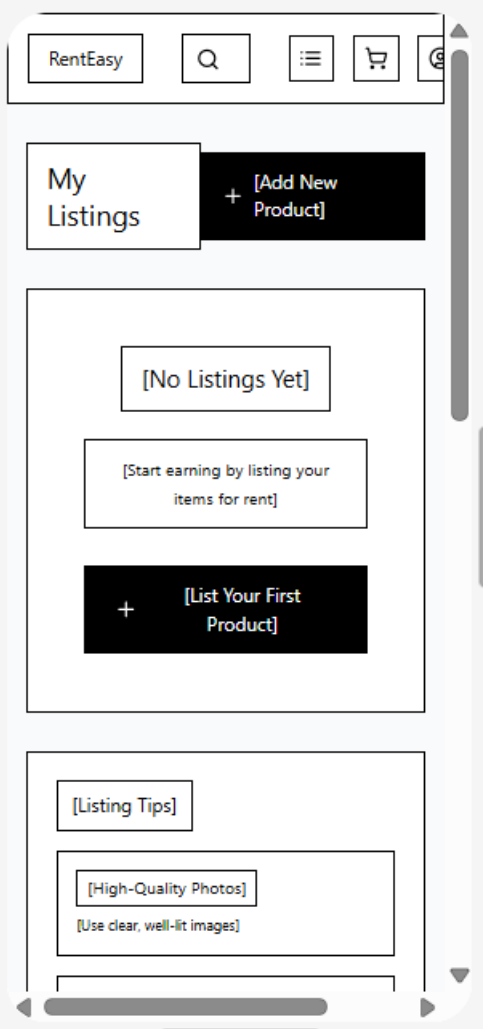
Login and Register Page



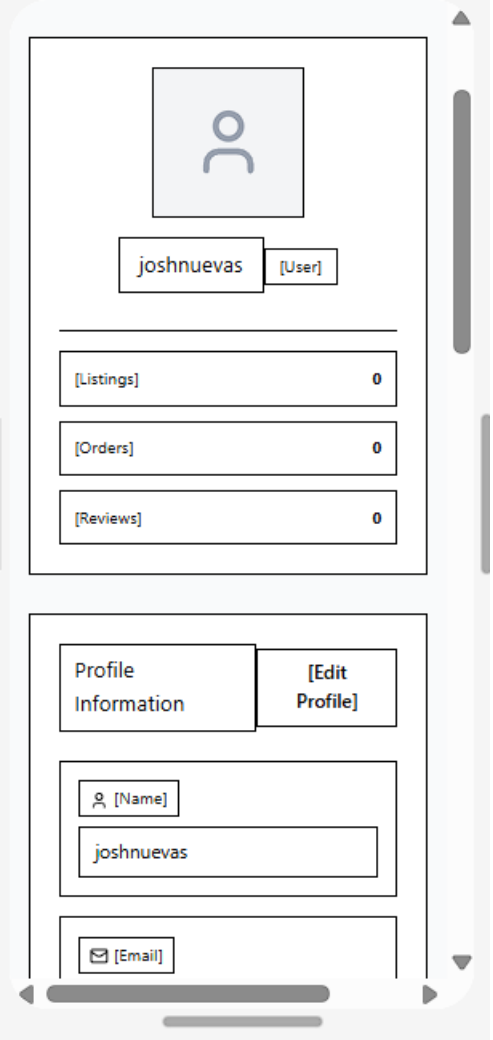


User Home Page

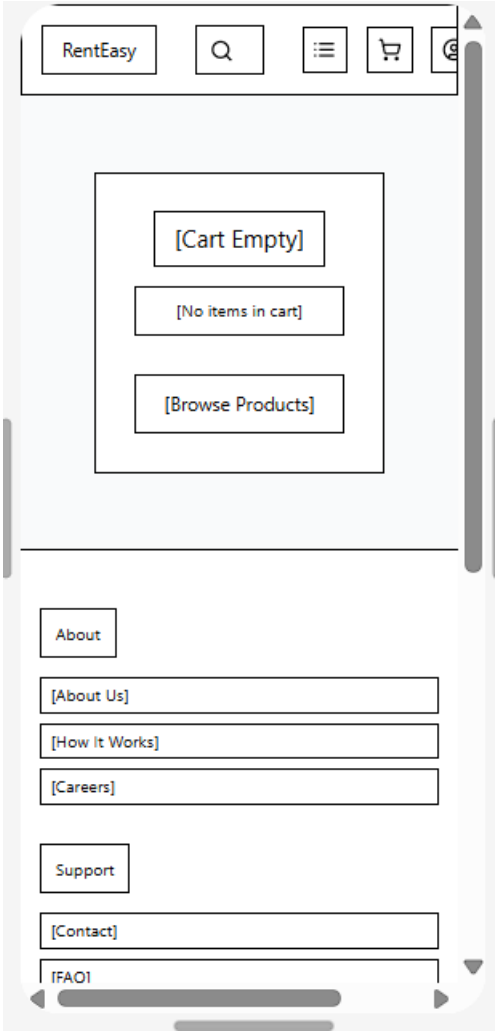
Create Listing Page



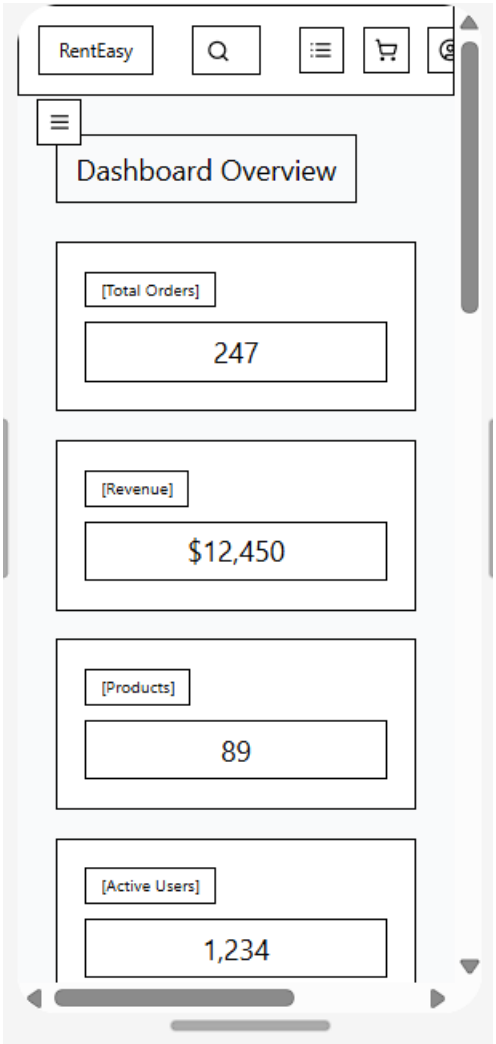
Cart Page

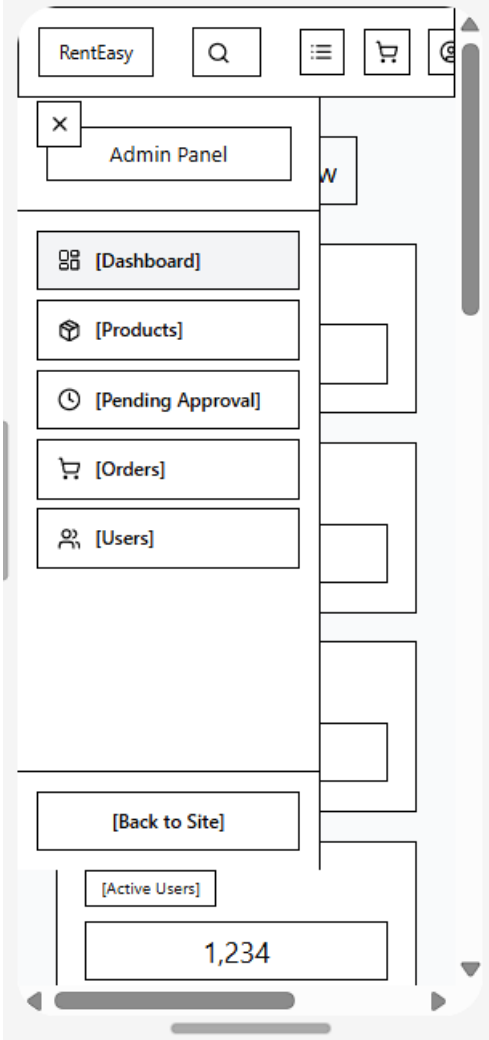


User Profile Page

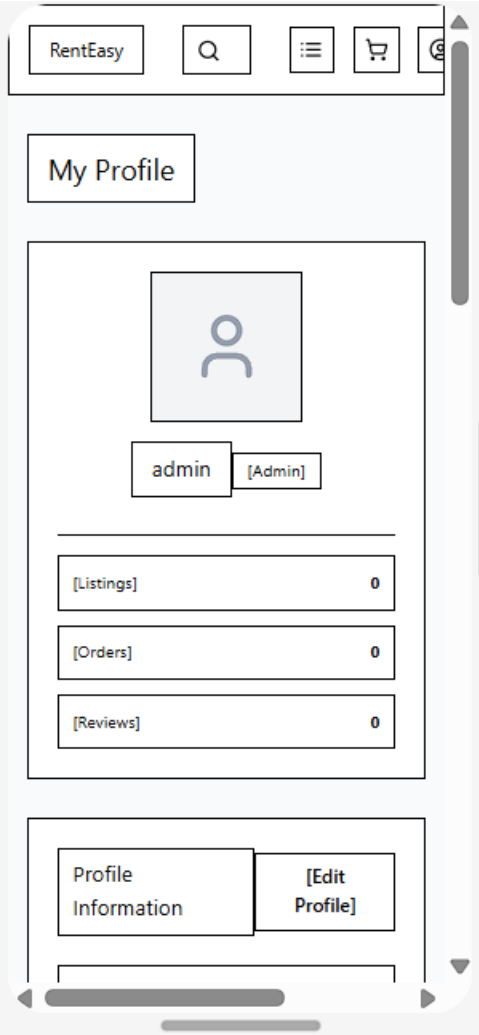


Admin Dashboard

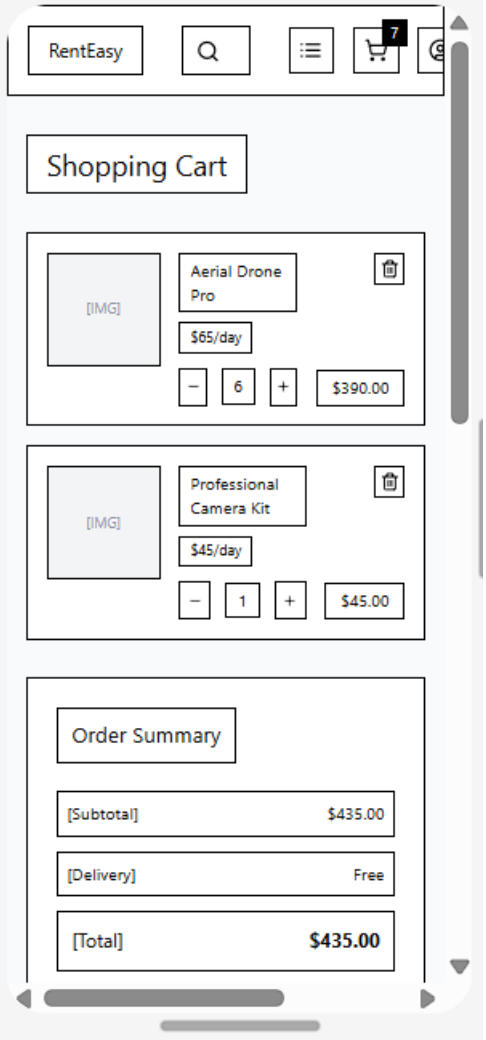


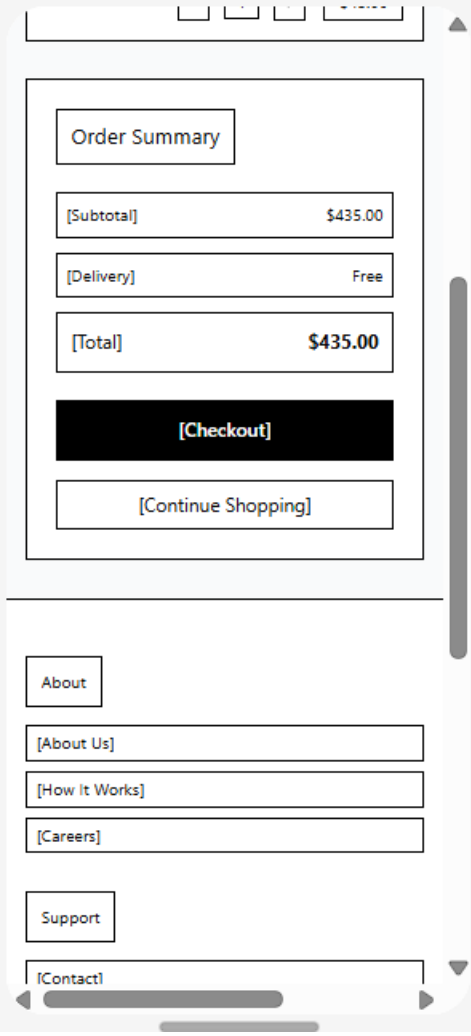


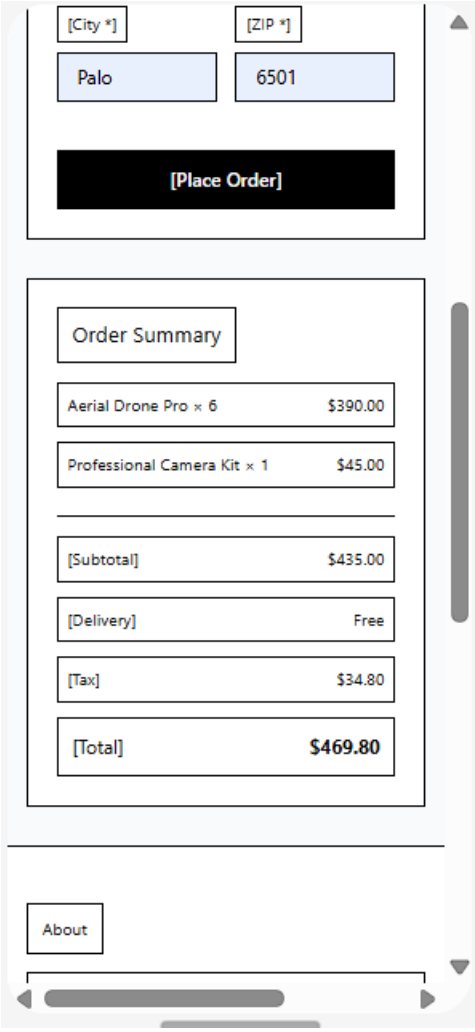
Admin Profile Page

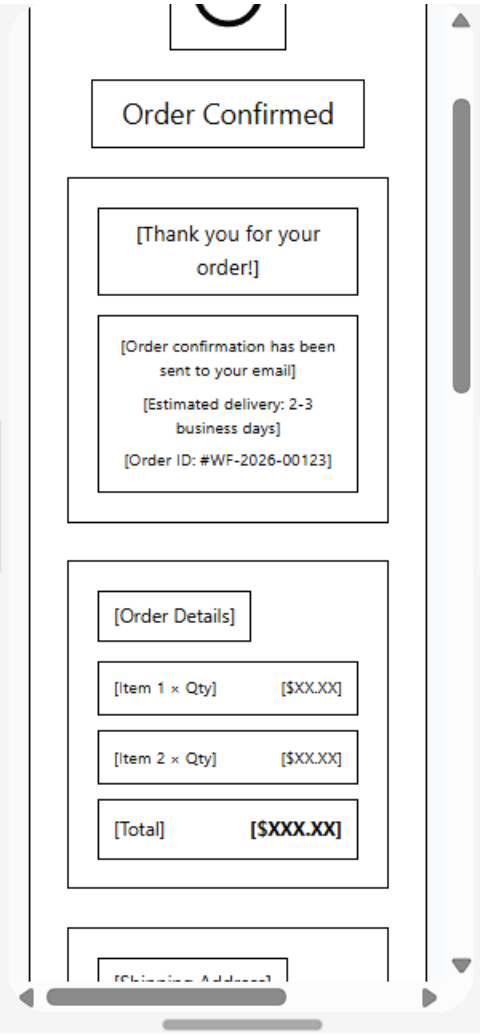


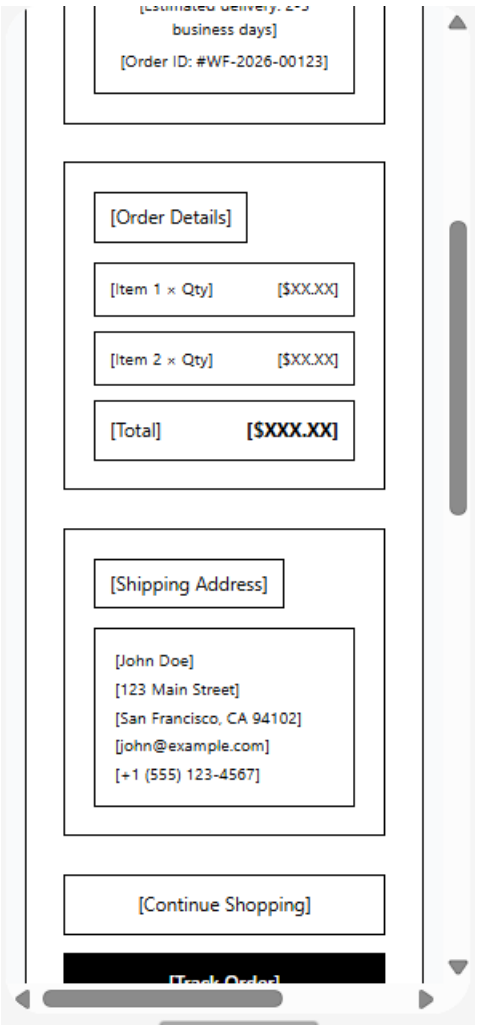
Payment Process Page




Order Summary
[Subtotal] $435.00
[Delivery] Free
[Total] $435.00
[Checkout]
[Continue Shopping]
About
[About Us]
[How It Works]
[Careers]
Support
[Contact]


[City *]
[ZIP *]
Palo
6501
[Place Order]
Order Summary
Aerial Drone Pro × 6 $390.00
Professional Camera Kit × 1 $45.00
[Subtotal] $435.00
[Delivery] Free
[Tax] $34.80
[Total] $469.80
About


Order Confirmed
[Thank you for your order!]
[Order confirmation has been sent to your email]
[Estimated delivery: 2-3 business days]
[Order ID: #WF-2026-00123]
[Order Details]
[Item 1 × Qty] [$XX.XX]
[Item 2 × Qty] [$XX.XX]
[Total] [$XXX.XX]


business days]
[Order ID: #WF-2026-00123]
[Order Details]
[Item 1 × Qty] [$XX.XX]
[Item 2 × Qty] [$XX.XX]
[Total] [$XXX.XX]
[Shipping Address]
[John Doe]
[123 Main Street]
[San Francisco, CA 94102]
[john@example.com]
[+1 (555) 123-4567]
[Continue Shopping]

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1–2)

Week 1: Requirements & Architecture
Day 1–2: Project setup and documentation
Day 3–4: Complete Functional and Non-Functional Requirements (FRS & NFR)
Day 5–7: System architecture design and technology stack confirmation

Week 2: Detailed Design
Day 1–2: API specification and endpoint design
Day 3–4: Database schema and ERD finalization
Day 5–6: UI/UX wireframes in Figma (Customer, Admin)
Day 7: Final review of design documents

Phase 2: Backend Development (Week 3–4)

Week 3: Foundation
Day 1: Spring Boot project setup and dependencies
Day 2: Database configuration (PostgreSQL / Supabase) and entity models
Day 3: JWT authentication and role-based access control
Day 4: User management APIs (Customer, Admin)
Day 5: Product management APIs (Create, Update, Delete, View)

Week 4: Core Features
Day 1: Shopping cart functionality
Day 2: Order management and checkout process
Day 3: Search and filtering implementation
Day 4: Error handling, validation, and logging
Day 5: API testing, validation, and documentation

Phase 3: Web Application (Week 5–6)

Week 5: Frontend Foundation
Day 1: React + TypeScript project setup
Day 2: Authentication pages (Login, Register)
Day 3: Product listing page with search and filters
Day 4: Product detail page
Day 5: Shopping cart interface

Week 6: Complete Web Features
Day 1: Checkout flow implementation
Day 2: Order history and confirmation pages

Day 3: Admin dashboard (Product & Order management)
Day 4: Responsive UI improvements and usability testing
Day 5: Frontend integration with backend APIs

Phase 4: Mobile Application (Week 7–8)

Week 7: Android Foundation
Day 1: Android Studio setup and project configuration
Day 2: Authentication screens implementation
Day 3: Product browsing and search features
Day 4: Shopping cart functionality
Day 5: API service integration using Retrofit

Week 8: Complete Mobile App
Day 1: Checkout flow and order confirmation
Day 2: Order management interface for customer
Day 3: UI polish and performance improvements
Day 4: Testing on emulator and physical devices
Day 5: APK build generation and documentation

Phase 5: Integration & Deployment (Week 9–10)

Week 9: Integration Testing
Day 1: End-to-end testing across backend, web, and mobile
Day 2: Bug fixing and performance optimization
Day 3: Security review and authentication testing
Day 4: Database and API performance testing
Day 5: Documentation updates

Week 10: Deployment
Day 1: Backend deployment (Railway / Heroku)
Day 2: Web application deployment (Vercel / Netlify)
Day 3: Mobile APK distribution for testing
Day 4: Final system validation
Day 5: Project submission and presentation preparation

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancement